

Dokumentacja techniczna projektu RentalsBD2

Chen Peng-Yin, Salmanowicz Weronika
Szozda Wojciech, Winiarczyk Piotr, Konarski Jan

22 czerwca 2026

BD2 Zespół 26 : <https://github.com/Nexonus/BD2-Rentals>

1 Wstęp

Celem niniejszego projektu było stworzenie aplikacji do wynajmowania rowerów i zakupu akcesoriów rowerowych. Poniższy raport przedstawia techniczne aspekty zaimplementowanego rozwiązania.

1.1 Stos technologiczny:

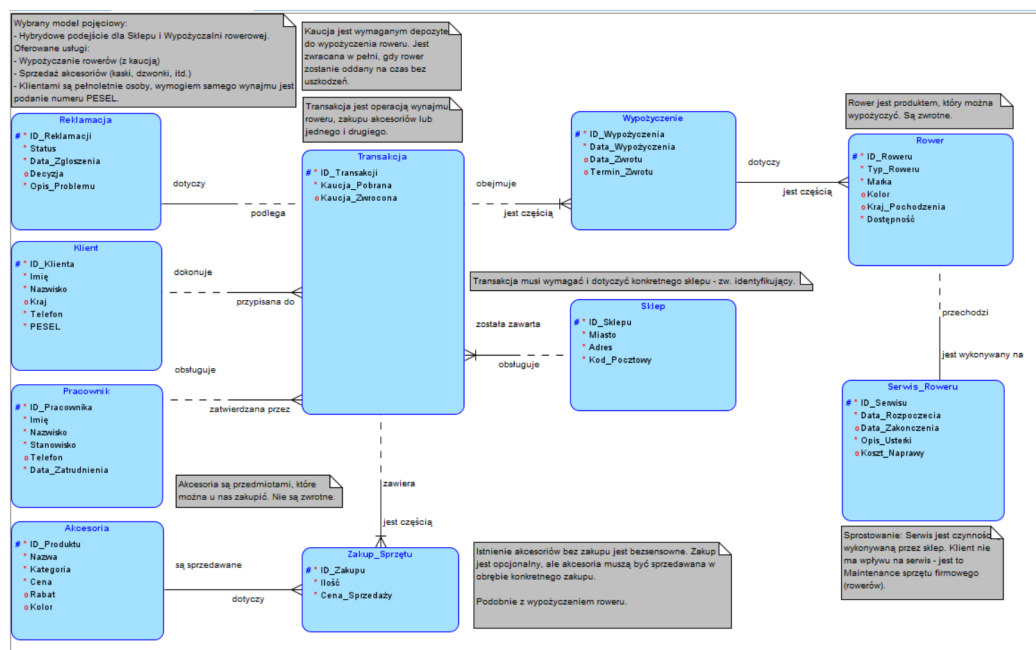
- Python 3.12
- Framework Django
- MySQL 8.0
- Visual Studio Code

2 Model ERD

Pierwszym z etapów projektu było stworzenie modelu ERD w Oracle Data Modeller, który zwizualizowałby logikę biznesową w projekcie wypożyczalni. W tym celu powstały następujące encje - **Transakcja** jest encją nadrzędną

naszego sklepu. Aby jej dokonać, ta musi zostać przypisana do konkretnej placówki w której się odbyła, może ale nie musi uwzględniać **Wypożyczenie** lub **Sprzedaż**. Warunkiem dokonania transakcji jest również przypisanie jej do konkretnego **Pracownika**, który ją obsługuje oraz **Klienta**, który jest użytkownikiem systemu.

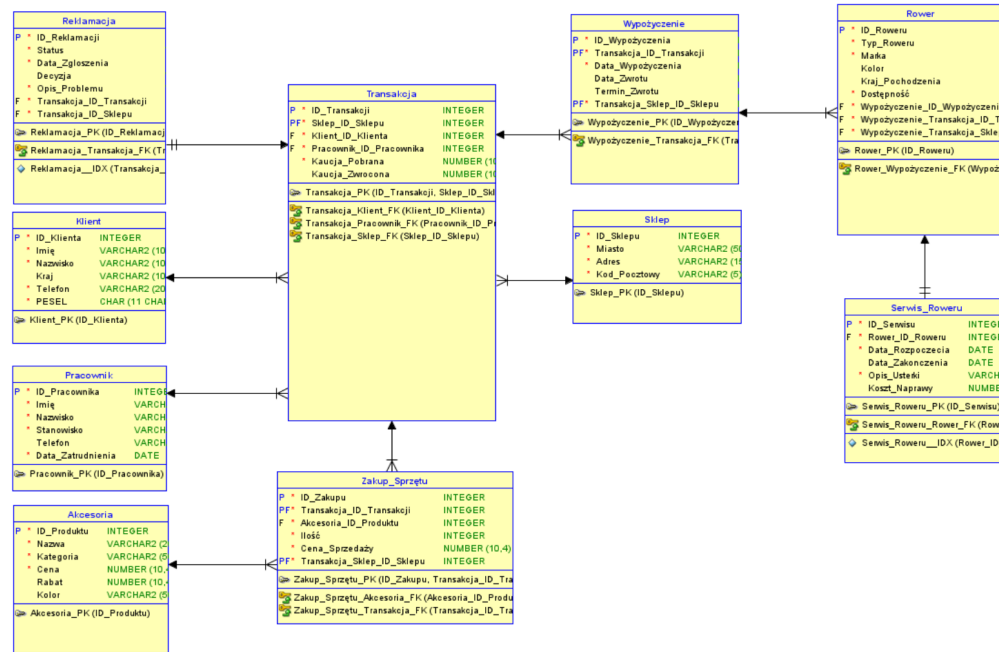
Terminy Sprzedaż i Wypożyczenie odróżniamy w następujący sposób - Sprzedaż jest udzielana jako jednorazowa transakcja uwzględniająca zakup co najmniej jednej kategorii przedmiotu w dowolnej ilości. Mianem Wypożyczenia definiujemy zakup usługi (w ramach zabrania roweru), która naliczana jest w stawce godzinowej w zależności od zdefiniowanego modelu.



Rysunek 1: Diagram ERD projektu.

Największym wyzwaniem w realizacji tego etapu było precyzyjne odzwierciedlenie logiki biznesowej w modelu danych. Po przeprowadzeniu kilku iteracji projektowych zdecydowaliśmy się na model hybrydowy, wykorzystujący encje pośrednie w celu lepszej wizualizacji i obsługi złożonych relacji między wypożyczeniami a sprzedażą sprzętu. Po sfinalizowaniu definicji modelu pojęciowego, przystąpiliśmy do opracowania jego postaci relacyjnej.

3 Model Relacyjny:



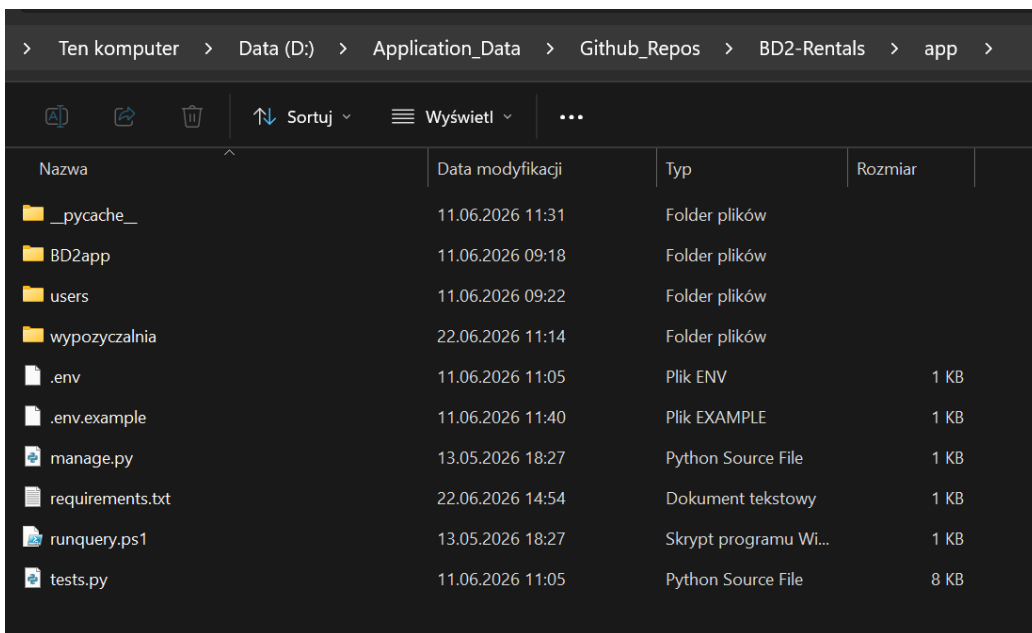
Rysunek 2: Model relacyjny dla projektu.

Struktura bazy danych (Rysunek 2) została zaprojektowana w sposób znormalizowany, co zapewnia integralność danych oraz optymalizuje procesy transakcyjne. Do kluczowych założeń projektu należą:

- **Centralizacja transakcji:** Encja Transakcja stanowi punkt wyjścia dla wszystkich operacji, integrując informacje o kliencie, pracowniku oraz punkcie sprzedaży (czyli sklepie).
- **Zarządzanie serwisem:** Relacja między Rower a Serwis_Roweru umożliwia śledzenie pełnej historii napraw oraz kosztów eksploatacji poszczególnych egzemplarzy.

- **Mechanizmy weryfikacji dostępności:** Zastosowanie mechanizmów bazodanowych pozwala na automatyczną weryfikację dostępności sprzętu, wykluczając rowery aktualnie znajdujące się w serwisie i wypożyczone. W podobny sposób sprawdzany jest stan dostępności asortymentu sklepu.
- **Elastyczność sprzedaży:** Relacje Wypożyczenie oraz Zakup_Sprzetu są rozwinięciem transakcji, co pozwala na obsługę scenariuszy hybrydowych - jednoczesnego wynajmu roweru oraz sprzedaży akcesoriów.
- **Integralność i spójność:** System wykorzystuje klucze obce oraz encje pośrednie, co eliminuje redundancję danych i wymusza logiczną poprawność wprowadzanych informacji.
- **Wydajność analityczna:** W przypadkach obliczania danych analitycznych (np. łączny utarg ze sprzedaży) zastosowano denormalizację poprzez wprowadzenie redundantnej wartości dla kosztów wynajmu roweru.

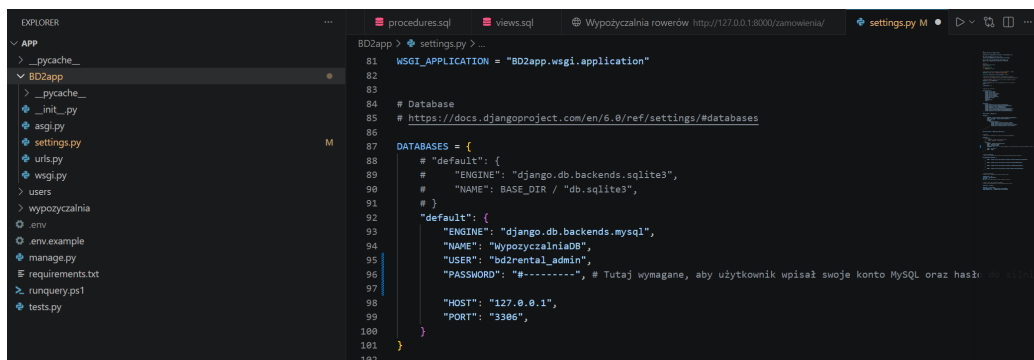
4 Model Fizyczny:



Nazwa	Data modyfikacji	Typ	Rozmiar
pycache	11.06.2026 11:31	Folder plików	
BD2app	11.06.2026 09:18	Folder plików	
users	11.06.2026 09:22	Folder plików	
wypożyczalnia	22.06.2026 11:14	Folder plików	
.env	11.06.2026 11:05	Plik ENV	1 KB
.env.example	11.06.2026 11:40	Plik EXAMPLE	1 KB
manage.py	13.05.2026 18:27	Python Source File	1 KB
requirements.txt	22.06.2026 14:54	Dokument tekstowy	1 KB
runquery.ps1	13.05.2026 18:27	Skrypt programu Wi...	1 KB
tests.py	11.06.2026 11:05	Python Source File	8 KB

Rysunek 3: Przechodzimy do katalogów App oraz BD2App po rozpakowaniu repozytorium.

Po rozpakowaniu repozytorium, możemy zarządzać aplikacją przechodząc do katalogu App. Aby rozpocząć pracę należy posiadać zainstalowany MySQL Server oraz MySQL Workbench w wersji 8.0. Następnie w katalogu BD2app otwieramy plik settings.py i ustawiamy dane administratora bazy danych oraz hasło do połączenia się z MySQL.



Rysunek 4: Ustawiamy USER oraz PASSWORD tak samo jak w naszej instancji MySQL na której zamierzamy uruchomić serwer. Aplikacja nie powinna być uruchamiana na silniku SQLite3 ze względu na brak wsparcia procedur składowych i różnice w składni wyzwalaczy!

Po skonfigurowaniu aplikacji można przystąpić do wykonywania pierwszej migracji. W tym celu należy wygenerować superużytkownika i tabele przy pomocy poleceń:

- `python manage.py makemigrations`
- `python manage.py migrate`
- `python manage.py createsuperuser`

Uwaga, ostatnie polecenie poprosi o ustawienie nazwy i hasła administratora usługi web. Nie jest wymagane podawanie adresu poczty e-mail. Po wykonaniu tych kroków w bazie MySQL powinny wygenerować się nowe tabele, co sprawdzimy przy pomocy polecenia kwerendy:

- `SHOW TABLES;`

84 • SHOW TABLES;

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [A](#)

Tables_in_wypożyczalniadb
django_session
view_analiza_sprzedazy
view_asortyment_sklep
view_przychod_rowery
view_przychody_miasta
view_statystyki
view_statystyki_pracownikow
wypożyczalnia_akcesoria
wypożyczalnia_klient
wypożyczalnia_koszyk
wypożyczalnia_pozycjakoszyka
wypożyczalnia_pracownik
wypożyczalnia_reklamacja
wypożyczalnia_rower
wypożyczalnia_serwisroweru
wypożyczalnia_sklep
wypożyczalnia_stanmagazynowyakcesoriow
wypożyczalnia_transakcja
wypożyczalnia_wypożyczenie
wypożyczalnia_zakupsprzetu

Result 5 x

Rysunek 5: Lista dostępnych tabel w projekcie BD2Rentals.

Oprócz widocznych tabel możliwe jest także wywołanie perspektyw analitycznych, w tym celu w katalogu Wypożyczalnia/Scripts umieszczono skrypty tworzące perspektywy, procedurę składową oraz wywołującą ją wyzwalacze.

```

1  --- Procedura składowa obliczająca łączny koszt z utargu zbuforowanego (SQL) + koszt zakupu sprzętu
2  --- Generalnie dzięki temu w zakładce Zamówienia poprawnie wyświetla się kwota do zapłaty uwzględniają
3  --- czas wynajmu / h
4  DELIMITER //
5
6  CREATE PROCEDURE przelicz_utarg_transakcji(p_transakcja_id INT)
7  BEGIN
8      DECLARE v_suma_wynajmy DECIMAL(12,2);
9      DECLARE v_suma_sprzedaz DECIMAL(12,2);
10
11      SELECT IFNULL(SUM(
12          CASE
13              WHEN termin_zwrotu IS NULL THEN cena_za_godzine
14              ELSE CEIL(TIMESTAMPDIFF(SECOND, data_wypozyczenia, termin_zwrotu) / 3600.0) * cena_za_godzine
15          END --- Ważne aby był CEIL bo inaczej będziemy zaokrąglali w dół różnicę godzinową (znikający koszt)
16          ), 0) INTO v_suma_wynajmy
17      FROM wypożyczalnia_wypozyczenie WHERE transakcja_id = p_transakcja_id;
18
19      SELECT IFNULL(SUM(cena_sprzedazy), 0) INTO v_suma_sprzedaz
20      FROM wypożyczalnia_zakupsprzetu WHERE transakcja_id = p_transakcja_id;
21
22      UPDATE wypożyczalnia_transakcja
23      SET utarg_calkowity_zbuforowany = v_suma_wynajmy + v_suma_sprzedaz
24      WHERE id = p_transakcja_id;
25  END //
26
27 DELIMITER ;

```

Rysunek 6: Procedura składowa obliczająca dynamicznie koszt wypożyczenia na bazie różnicy czasu oraz agregującą go do kosztu całkowitego transakcji.

Query 1 SQL File 1* SQL File 2* SQL File 3*

```

100  WHEN w.termin_zwrotu IS NULL THEN w.cena_za_godzine
101  ELSE CEIL(TIMESTAMPDIFF(SECOND, w.data_wypozyczenia, w.termin_zwrotu) / 3600.0) * w.cena_za_godzine
102  END
103  ) AS 'Przychód całkowity'
104  FROM wypożyczalnia_sklep s
105  JOIN wypożyczalnia_rower r ON s.id = r.sklep_id
106  JOIN wypożyczalnia_wypozyczenie w ON r.id = w.rower_id
107  GROUP BY s.miasto
108  ORDER BY `Przychód całkowity` DESC;
109
110  • SELECT * FROM view_przychody_miasta;

```

	Miasto	Adres	Marka	Typ	Nr Seryjny	Status
▶	Kra	ul. Krzywa 34	Giant	ROAD	R-ROAD-001	Dostępny
	Kra	ul. Krzywa 34	Kross	CITY	R-CITY-002	Dostępny
	Kra	ul. Krzywa 34	Merida	MTB	R-MTB-002	Dostępny
	Waw	ul. Prosta 12	Romet	CITY	R-CITY-001	Wypożyczony
	Waw	ul. Prosta 12	Trek	MTB	R-MTB-001	Wypożyczony

view_analiza_sprzedazy 1 view_statystyki2 view_przychod_rowery 3 view_statystyki_pracownikow 4 view_asortyment_sklep 5 x view_przychody_miasta 6 Read Only

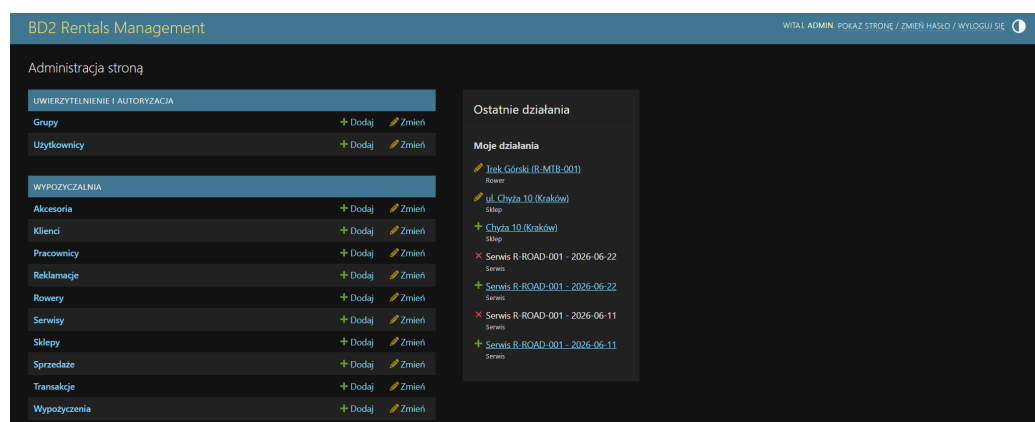
Rysunek 7: Przykłady perspektyw analitycznych - widok dostępnego asortymentu.

W projekcie dostępne są następujące perspektywy:

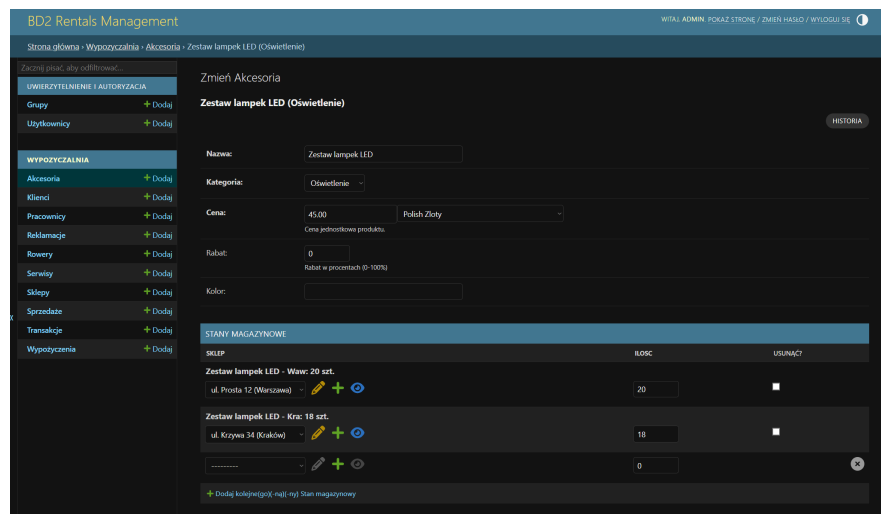
- Analiza przychodu ze sprzedaży określonych kategorii produktów (np. łączny przychód ze sprzedaży kasków).
- Statystyki czasu wypożyczenia rowerów (jak długo dany był wypożyczony, popularność produktu).
- Przychód i liczba wypożyczeń rowerów.
- Statystyki pracowników, ile transakcji obsłużyli.
- Przychody zagregowane ze sklepów w danym mieście.

5 Panel administratorski

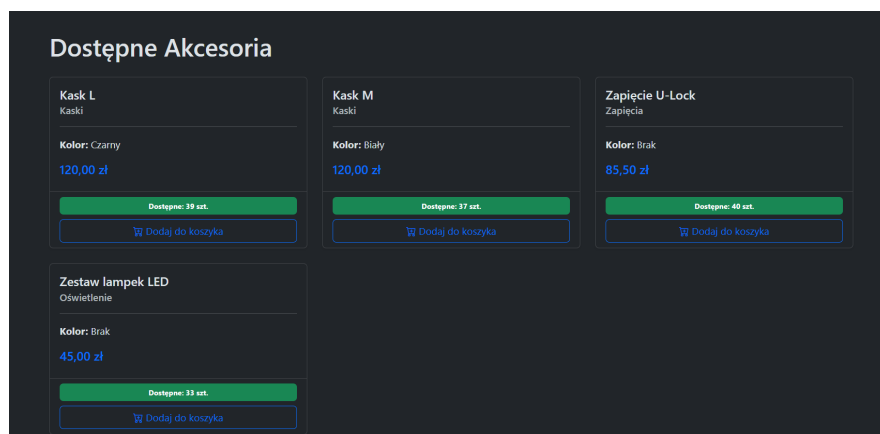
W celu przejścia do sekcji dla administracji usługi WWW należy przejść pod adres lokalnej instancji 127.0.0.1:8000/admin.



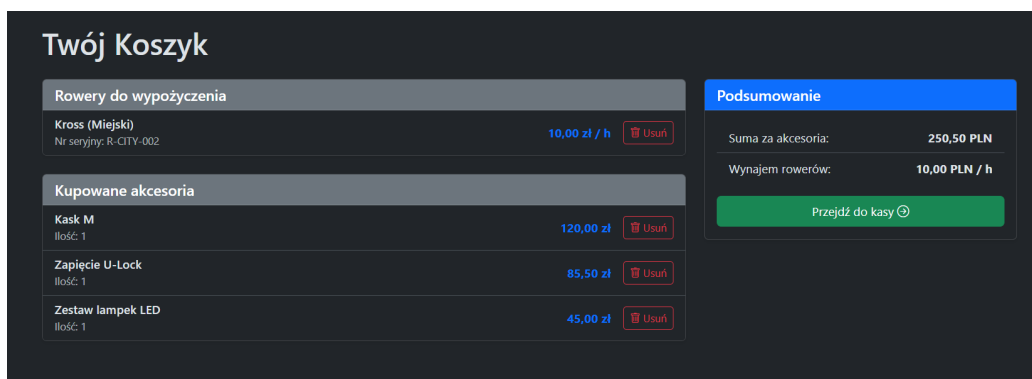
Rysunek 8: Panel administratorski pozwala na zarządzanie danymi dostępnymi w sieci sklepów.



Rysunek 9: Klikając w tabelę Akcesoria możliwe jest dostosowanie liczby przedmiotów znajdujących się w danym sklepie. Dla uproszczenia problemu - uznajemy, że akcesoria liczymy dla całej sieci. Kupując lampki LED wpierw wyczerpujemy asortyment pierwszego stanu magazynowego na liście.



Rysunek 12: Dostępność akcesoriów jak i rowerów zmienia się dynamicznie w aplikacji. Zmiana statusu roweru na Niedostępny, bądź dodanie go do Serwisu sprawia, że ten znika z listy.



Rysunek 13: Widok koszyka z perspektywy użytkownika.

Moje Zamówienia

Zalogowano jako: Test User

Transakcja #34

22 Cze 2026, 16:16

Wypożyczone rowery

Kross Miejski

Nr: R-CITY-002 | Zarezerwowany od: 16:16

10,00 zł / godz.

Zwróć rower

Kupiony sprzęt

Kask M

1 szt.

120,00 zł

Zapięcie U-Lock

1 szt.

85,50 zł

Zestaw lampek LED

1 szt.

45,00 zł

Kaucja pobrana: 0,00 zł

Kaucja zwrócona: 0,00 zł

Koszt całkowity: 260,50 zł

Zgłoś reklamację

Transakcja #33

22 Cze 2026, 16:15

Transakcja #32

22 Cze 2026, 16:07

Transakcja #31

22 Cze 2026, 16:01

Transakcja #30

22 Cze 2026, 16:00

Rysunek 14: Finalizacja transakcji i możliwość zwrotu roweru. Uznajemy, że na tym etapie doszło do sfinalizowania transakcji z użytkownikiem serwisu. Zwrot roweru po czasie skutkuje naliczeniem dodatkowych opłat liczonych dla minimalnej stawki godzinowej. Wraz z przekroczeniem 60 minut naliczana jest kolejna opłata.

```

def clean(self):
    if not self.pk and self.rower and not self.rower.dostepnosc:
        raise ValidationError({'rower': "This bike is unavailable for rental."})

def save(self, *args, **kwargs):
    self.full_clean()
    if not self.pk:
        self.cena_zagodzine = self.rower.cena_zagodzine # Denormalizacja - kopiujemy wartość
        self.rower.status(False)
    super().save(*args, **kwargs)

def __str__(self):
    return f"WY-{self.id:04d}"

class Rower(models.Model):
    MODEL_CHOICES = [
        ('MTB', 'Górski'),
        ('ROAD', 'Szosowy'),
        ('CITY', 'Miejski')
    ]

```

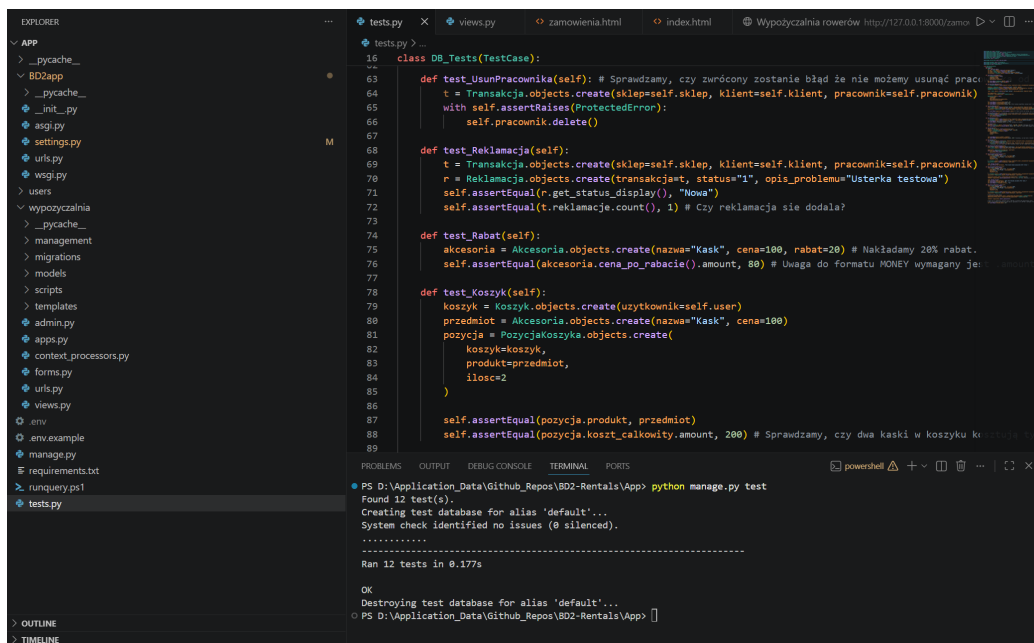
Rysunek 15: Denormalizacja danych - przykład dla kolumny agregującej.

```

PS D:\Application_Data\Github_Repos\BD2-Rentals\App> py .\manage.py insert_data
Dodawanie sklepów...
Dodawanie pracowników...
Dodawanie klientów...
Dodawanie akcesoriów...
Dodawanie stanów magazynowych...
Dodawanie rowerów...
Dodawanie przykładowych transakcji...
Pomyślnie dodano dane do bazy danych
PS D:\Application_Data\Github_Repos\BD2-Rentals\App>

```

Rysunek 16: W celu wgrania danych testowych dla aplikacji możliwe jest wykorzystanie poleceń: python manage.py insert_data oraz wypelnij_baze.



Rysunek 17: Testy jednostkowe aplikacji mogą zostać przeprowadzone przy pomocy polecenia `python manage.py test`.